

Grapher Implementation Addendum: Provenance and Future Training Readiness

Continue implementing the existing Grapher specification as written. Do not redesign the architecture around machine learning, an LLM, or a future training pipeline.

However, ensure that Grapher's schema and mutation model preserve enough structured provenance to reconstruct meaningful state transitions later.

Objective

Grapher must be able to answer both:

1. What is the current/canonical state of the work graph?
2. How did that state come to exist?

The second requirement is important for auditability, retrospectives, workflow analysis, and future downstream uses of Grapher history.

Preserve State Transitions

When meaningful graph state changes occur, Grapher should preserve a structured historical record of the transition rather than only replacing the old value.

Examples include:

- workflow-state changes
- verification-state changes
- node status changes
- lifecycle-stage changes
- decisions being proposed, accepted, rejected, superseded, or overridden
- requirements being added, changed, satisfied, invalidated, or superseded
- evidence being attached or invalidated
- blockers appearing or resolving
- dependencies being added, removed, or superseded
- incidents and deviations
- checkpoint creation, refresh, or invalidation
- canonical/current truth changing

A transition record should preserve, where applicable:

- stable event/transition ID
- affected node/entity ID
- event/transition type

- previous state/value
- resulting state/value
- timestamp
- actor/source responsible for the change
- reason or rationale, when supplied
- evidence references
- related decision IDs
- related requirement IDs
- supersession/override relationships
- correlation or operation ID when multiple changes belong to the same action

Actor/source information should be able to distinguish at minimum between:

- human
- agent
- system/tool
- migration/import

Do not require an LLM-specific actor model.

Outcome Must Remain Distinct From Decision

Do not collapse a decision and its eventual outcome into the same field.

For example:

Decision: `REQUEST_VERIFICATION`

Later outcome: `verification_failed`

Later still: `implementation_revised`

Final outcome: `verification_passed`

These should remain reconstructable as separate events.

This is important because later analysis must be able to compare what was believed or decided at one point in time against what subsequent evidence established.

Preserve Proposed vs Actual

Where relevant, maintain the distinction between:

- proposed action
- executed action
- observed result

- verified result
- final/canonical state

Do not silently convert one into another.

Human/Agent Overrides

If a previously proposed or executed decision is overridden, preserve:

- the original decision
- the overriding decision
- who/what performed the override
- supplied rationale
- resulting state

Do not delete the original record.

Evidence Provenance

Evidence references should remain attributable to their source when possible.

Examples:

- test result
- document
- command output
- human observation
- agent report
- external artifact
- Grapher-derived state

An agent assertion should not automatically become equivalent to independently verified evidence.

Current-State Performance

This historical/provenance requirement must not require callers to reconstruct the entire event history simply to determine current graph state.

Grapher may maintain efficient canonical/current representations while retaining transition history separately.

In other words:

`current state` and `historical provenance` are complementary representations.

Backward Compatibility

Maintain all compatibility requirements from the primary implementation specification, including existing keyed-node behavior, migration requirements, and preservation of existing relations/data.

Do not introduce a breaking schema change merely to implement this addendum.

Do Not Build Training Functionality Yet

Do NOT:

- fine-tune a model
- add model-specific fields
- create LLM prompts
- make assumptions about a particular model architecture
- turn Grapher into an ML pipeline
- automatically assign "good decision" / "bad decision" labels
- store enormous conversational transcripts merely for possible future ML use

Grapher's responsibility is to preserve accurate, structured operational history.

A separate downstream system can later transform selected historical records into datasets.

The architectural principle is:

Grapher records facts, states, transitions, provenance, evidence, and outcomes.

Future systems decide how those records are interpreted or used.